

Detecção de Cheaters no Cs:Go Usando Aprendizado de Máquina

João Augusto Paixão Rocha

UNIVERSIDADE FEDERAL

UNIFESP

São José dos Campos, Brasil

joao.paixao@unifesp.br

Abstract—Este trabalho apresenta uma abordagem de aprendizado de máquina para a detecção de cheaters do tipo aimbot no jogo Counter-Strike: Global Offensive (CS:GO), inserida no contexto das ODS 8 e 9 ao contribuir para a promoção de ambientes digitais mais justos e confiáveis. Foi utilizado um conjunto de dados com 12.000 jogadores, sendo 2.000 cheaters confirmados e 10.000 jogadores legítimos, no qual cada jogador é representado por 30 rounds de 192 ticks, com 5 características comportamentais registradas a cada tick. Três algoritmos foram avaliados por meio de validação cruzada estratificada com 5 folds: um Random Forest, operando sobre 100 features estatísticas extraídas por jogador; uma Rede Neural Convolucional 1D (CNN), operando sobre a série temporal bruta subamostrada; e uma LSTM Bidirecional, operando sobre sequências de features agregadas por round.

I. INTRODUÇÃO

Diante da crescente popularização dos jogos eletrônicos competitivos, a integridade das partidas tornou-se um tema que exige atenção cada vez maior por parte das desenvolvedoras e da comunidade acadêmica. Jogos de tiro em primeira pessoa (FPS) como o Counter-Strike: Global Offensive (CS:GO) movimentam uma indústria multibilionária que depende diretamente da percepção de jogo justo para manter o engajamento dos jogadores. Entre as formas de trapaça mais prejudiciais está o aimbot, um software que controla automaticamente a mira do jogador com uma precisão impossível de ser replicada por um humano, comprometendo tanto a experiência de jogadores legítimos quanto a viabilidade de competições profissionais de e-Sports. Os sistemas anti-cheat atualmente empregados pela indústria, como o Valve Anti-Cheat (VAC), baseiam-se majoritariamente na detecção por assinatura de softwares de trapaça conhecidos. Essa abordagem falha diante de variantes novas ou ofuscadas, e não é capaz de identificar anomalias comportamentais causadas por aimbots que não realizam injeção de código no processo do jogo, como é o caso dos aimbots baseados em visão computacional. Isso motiva a aplicação de aprendizado de máquina sobre dados comportamentais, em que a pergunta deixa de ser qual software está em execução e passa a ser como o jogador se move. Este trabalho está inserido no contexto dos ODS 8 e 9 da ONU, que defendem a promoção do crescimento econômico sustentável e da inovação em infraestruturas tecnológicas. Entende-se que ambientes digitais constituem uma forma contemporânea de ecossistema econômico e infraestrutura, e

que ferramentas automatizadas e escaláveis para a detecção de comportamento desleal contribuem diretamente para esses objetivos, ao garantir que a viabilidade profissional dos e-Sports e a segurança da plataforma sejam preservadas. Neste trabalho, foram comparadas três abordagens de aprendizado de máquina Random Forest, CNN 1D e LSTM Bidirecional, cada uma projetada para capturar anomalias comportamentais em uma escala temporal diferente, avaliadas sobre um conjunto de dados realista e desbalanceado de telemetria de jogadores de CS:GO.

II. TRABALHOS RELACIONADOS

A. Behavioral-based cheating detection in online first person shooters using machine learning techniques

Uma das abordagens pioneiras na detecção focada no lado do servidor para jogos FPS utilizou Máquinas de Vetores de Suporte (SVM) baseadas em características comportamentais extraídas de logs das partidas. A grande contribuição desse estudo foi estabelecer que é possível identificar trapaças apenas analisando os dados de entrada, eliminando completamente a necessidade de exigir acesso à máquina do cliente ou realizar invasivas varreduras de memória.

B. Deep learning and multivariate time series for cheat detection in video games

Avançando para a modelagem do movimento físico, outro trabalho reformulou as sequências de interação de mouse e teclado como séries temporais multivariadas. Ao aplicar uma Rede Neural Convolucional (CNN 1D) para classificar especificamente comportamentos de triggerbot e aimbot no CS:GO, o modelo alcançou uma acurácia média superior a 0.98, mesmo avaliando jogadores nunca vistos durante o treinamento. Essa eficácia na extração de padrões temporais locais foi o que motivou diretamente a escolha e a arquitetura da CNN adotada neste projeto.

C. BotScreen: Trust everybody, but cut the aimbots yourself

Com foco em ambientes de larga escala, o sistema distribuído BotScreen explorou o uso de uma rede neural recorrente combinada com um sistema de pontuação de anomalia. O modelo foca em rastrear padrões comportamentais anormais gerados pela assistência de mira, com a vantagem arquitetural de operar de forma independente, sem exigir qualquer modificação no motor do jogo.

O cenário de detecção, no entanto, é dinâmico e cada vez mais complexo. Uma pesquisa recente demonstrou como redes adversariais generativas (GANs) podem ser utilizadas para treinar aimbots com movimentos incrivelmente parecidos com os de um ser humano. Esses bots conseguiram enganar tanto os algoritmos de detecção convencionais quanto os próprios avaliadores humanos. Esse estudo evidencia a natureza adversarial do problema: os trapaceiros estão evoluindo, o que torna inegociável a necessidade de desenvolvermos modelos comportamentais mais robustos.

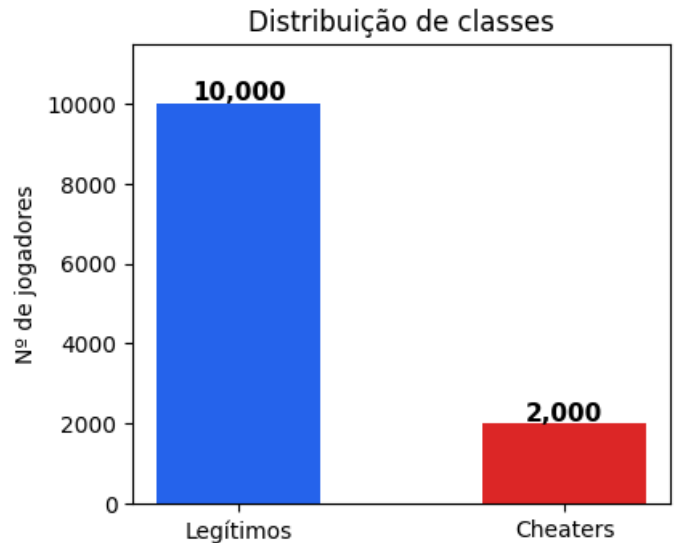


Fig. 1. Distribuição de Classes dos Players

E. HAWK: An anti-cheat framework for CS:GO

Por fim, o framework multi-view HAWK tentou uma abordagem focada em reproduzir o processo cognitivo e a tomada de decisão de especialistas humanos ao avaliar suspeitos. Esse sistema foi avaliado sobre o primeiro grande conjunto de dados real contendo múltiplos tipos de trapagens e conseguiu superar o anti-cheat oficial utilizado pela desenvolvedora, apresentando tanto uma taxa de detecção (recall) superior quanto tempos de banimento consideravelmente mais curtos.

III. CONJUNTO DE DADOS E FEATURES

O conjunto de dados é composto por 12.000 jogadores de CS:GO: 2.000 cheaters confirmados e 10.000 jogadores legítimos, resultando em uma razão de desbalanceamento de 1:5. Cada jogador é representado por um tensor de quatro dimensões com formato (30, 192, 5), correspondendo a 30 rounds, 192 ticks de servidor por round e 5 features comportamentais por tick. Sendo assim, o formato é (12000, 30, 192, 5). As cinco features registradas a cada tick são: AbsDeltaYaw (ADY), que representa a variação horizontal absoluta do ângulo de mira; AbsDeltaPitch (ADP), que representa a variação vertical absoluta do ângulo de mira; Crosshair-ToVictimYaw (CTVY) e Crosshair-ToVictimPitch (CTVP), que representam o erro angular entre a mira do jogador e o inimigo mais próximo; e um indicador binário de disparo, Fire (F), que indica se um tiro foi efetuado naquele tick.

Em vez de reamostrar o conjunto de dados, o desbalanceamento entre as classes foi tratado por meio da atribuição de pesos para cada classe durante o treinamento: peso 1,0 para jogadores legítimos e 5,0 para cheaters. Essa abordagem é matematicamente equivalente a um oversampling da classe minoritária, porém evita os riscos de data leakage (vazamento de dados) associados à duplicação de amostras antes da divisão da validação cruzada.

A partir do tensor bruto, foram derivadas três representações de entrada distintas, uma para cada algoritmo. Para o Random Forest, foram extraídas 100 features estatísticas por jogador: 15 estatísticas por feature bruta (média, desvio padrão, mínimo, máximo, mediana, skewness, curtose, percentis 25 e 75, amplitude, variação média absoluta entre ticks, desvio padrão das variações, variação máxima absoluta, percentil 95 das variações absolutas e taxa de picos acima de 3 desvios padrão), calculadas sobre os 5.760 ticks totais, somadas a 25 agregados por round. Para a CNN, a sequência bruta foi achatada para o formato (5.760, 5) e subamostrada por um fator de 4, resultando em (1.440, 5). Para a LSTM, cada um dos 30 rounds foi resumido em um vetor de 25 dimensões (média, desvio padrão, máximo, mínimo e amplitude de cada uma das 5 features), gerando uma sequência de formato (30, 25).

Antes do treinamento dos modelos, foi conduzida uma análise exploratória sobre o conjunto de dados bruto, da qual resultaram cinco figuras:

os histogramas de distribuição das cinco features, calculados sobre todas as amostras;

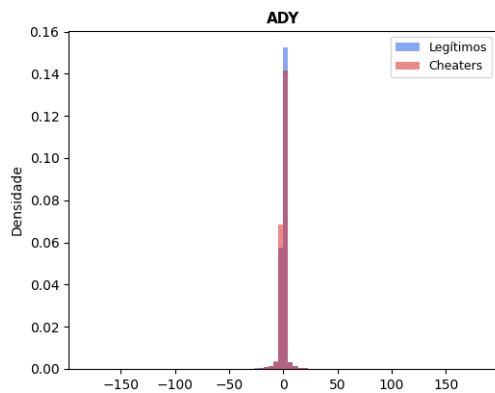


Fig. 2. Histograma da feature ADY

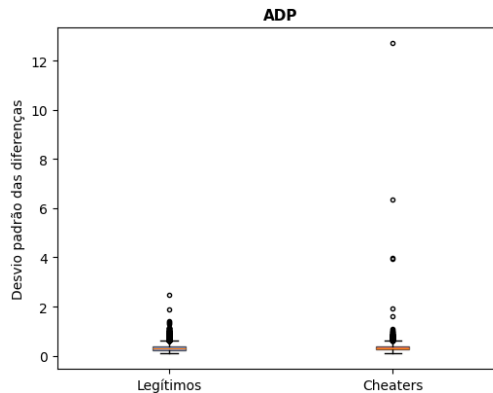


Fig. 5. Boxplot da feature ADP

dispersões por jogador relacionando pares de features;

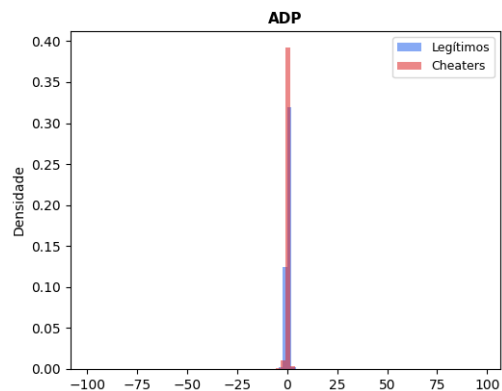


Fig. 3. Histograma da feature ADP

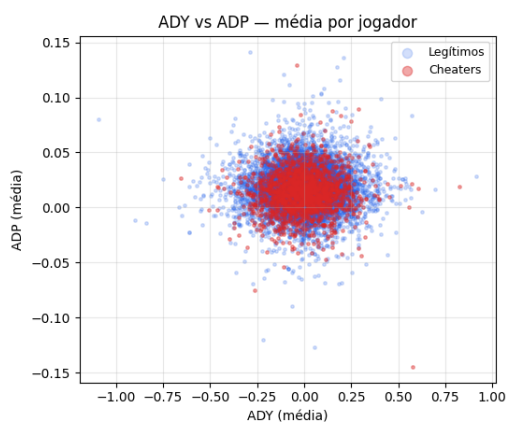


Fig. 6. Dispersão das features ADP e ADY

boxplots da variabilidade Delta-tick de cada feature, medindo o quão brusco foi o movimento;

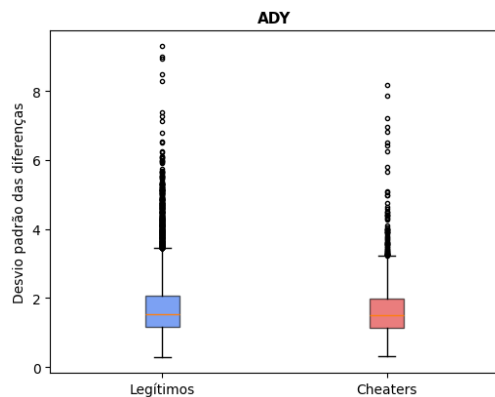


Fig. 4. Boxplot da feature ADY

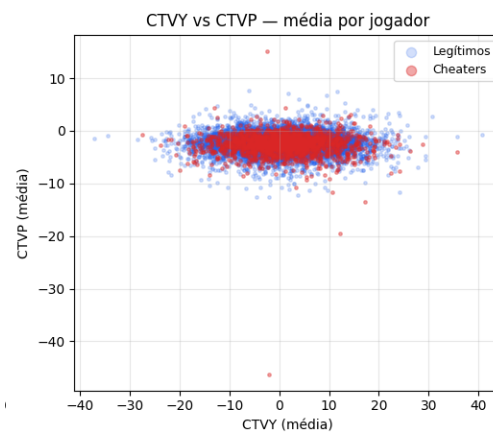


Fig. 7. Dispersão das features CTVY e CTVP

e séries temporais médias por feature, com faixa de ± 1 desvio padrão entre jogadores:

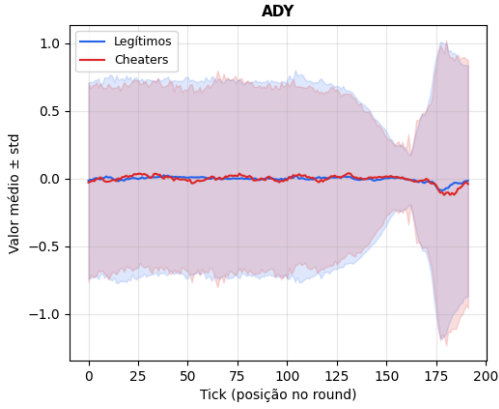


Fig. 8. Série Temporal da feature ADY

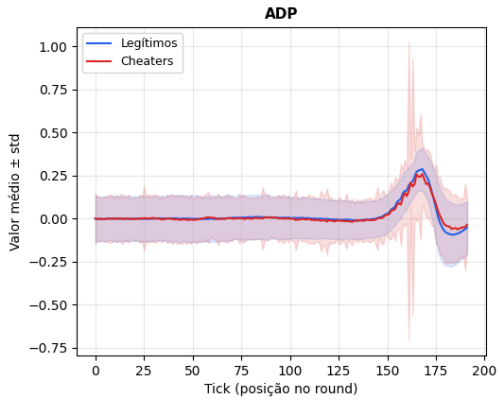


Fig. 9. Série Temporal da feature ADP

Analizando os dados gerados, incluindo os mostrados, chegamos a conclusão de que apesar de ambos grupos terem um perfil de dados muito parecido, os cheaters, em média, tem dados mais extremos. Os histogramas mostram a variação horizontal maior no grupo dos legítimos, enquanto a variação vertical é maior no grupo dos cheaters. Os boxplots mostram a mesma história, onde o ADP (movimento vertical) tem muito mais outliers, e esses são muito mais extremos. As dispersões mostram o mesmo formato, mas a dispersão dos cheaters é maior para todas as direções.

IV. METODOLOGIA

Todos os modelos foram avaliados por meio de Validação Cruzada Estratificada com 5 folds (Stratified 5-Fold Cross-Validation), garantindo que a razão de 1:5 entre as classes fosse preservada em cada fold. A normalização dos dados de entrada foi aplicada estritamente dentro de cada fold, utilizando apenas estatísticas calculadas a partir da partição de treino, o que evita qualquer vazamento de informação do conjunto de teste para o processo de treinamento.

Para converter as probabilidades nas classes finais, o limiar padrão de 0,5 foi descartado. O corte de cada modelo foi

calibrado na partição de treino de cada fold, maximizando a métrica F-beta com beta igual a 0,5. Isso foi feito para priorizar a precisão em detrimento do recall. Na prática, essa medida reduz os falsos positivos e evita o banimento incorreto de jogadores inocentes, que é a maior preocupação para um sistema deste num cenário real.

A. Random Forest

O classificador Random Forest foi treinado com 200 estimadores, profundidade máxima de 15 e um mínimo de 10 amostras por folha, utilizando pesos de classe balanceados. A profundidade relativamente rasa, aliada à regularização por folha, evita o overfitting em relação à classe minoritária, sem limitar excessivamente a capacidade do modelo de separar o espaço de 100 features. A importância das features foi extraída a partir do último fold, permitindo identificar quais assinaturas estatísticas mais discriminam cheaters de jogadores legítimos.

B. CNN

A CNN recebe como entrada a série temporal subamostrada, de formato (1.440, 5). A arquitetura é composta por três blocos convolucionais, cada um contendo uma camada Conv1D com kernel de tamanho 7 e ativação ReLU, seguida de Batch Normalization, MaxPooling1D com tamanho de pool 4 e Dropout. O número de filtros dobra a cada bloco (64, 128, 256), e os mapas de features são agregados por meio da concatenação entre GlobalAveragePooling1D e GlobalMaxPooling1D antes da camada de classificação. Essa concatenação permite que o modelo capture tanto o perfil comportamental típico do jogador quanto o momento mais anômalo isolado da partida — sendo esse último particularmente importante para a detecção de aimbots que produzem picos de precisão isolados e de curta duração. O treinamento utilizou o otimizador Adam com taxa de aprendizado de $1e-3$, função de perda binary cross-entropy, early stopping com paciência de 5 sobre o AUC de validação, e redução da taxa de aprendizado com paciência de 3.

C. LSTM

A LSTM recebe como entrada a sequência de features por round, de formato (30, 25). A arquitetura é composta por duas camadas LSTM Bidirecionais: a primeira com 64 unidades retornando sequências, e a segunda com 32 unidades. Cada camada é seguida de Dropout. A formulação bidirecional permite que o modelo capture dependências temporais entre rounds em ambas as direções, possibilitando, por exemplo, a detecção de um comportamento anormalmente consistente ao longo de toda a partida, ou de mudanças comportamentais em pontos específicos do jogo. Uma camada Dense com 64 unidades precede a saída sigmoide.

V. RESULTADOS

A. Resultados Individuais

O Random Forest obteve os resultados abaixo:

TABLE I
MÉTRICAS DE AVALIAÇÃO DO RANDOM FOREST

Classe / Métrica	Precision	Recall	F1-score	Support
Legítimo	0.86	0.94	0.90	2000
Cheater	0.46	0.26	0.33	400
Accuracy			0.82	2400
Macro avg	0.66	0.60	0.62	2400
Weighted avg	0.80	0.82	0.80	2400

A CNN obteve os resultados abaixo:

TABLE II
MÉTRICAS DE AVALIAÇÃO DA CNN

Classe / Métrica	Precision	Recall	F1-score	Support
Legítimo	0.89	0.95	0.92	2000
Cheater	0.64	0.40	0.49	400
Accuracy			0.86	2400
Macro avg	0.76	0.68	0.70	2400
Weighted avg	0.85	0.86	0.85	2400

E a LSTM obteve os resultados abaixo:

TABLE III
MÉTRICAS DE AVALIAÇÃO DO LSTM

Classe / Métrica	Precision	Recall	F1-score	Support
Legítimo	0.86	0.92	0.89	2000
Cheater	0.36	0.24	0.29	400
Accuracy			0.80	2400
Macro avg	0.61	0.58	0.59	2400
Weighted avg	0.78	0.80	0.79	2400

B. Resultados gerais

Desta forma, obtemos os seguintes resultados:

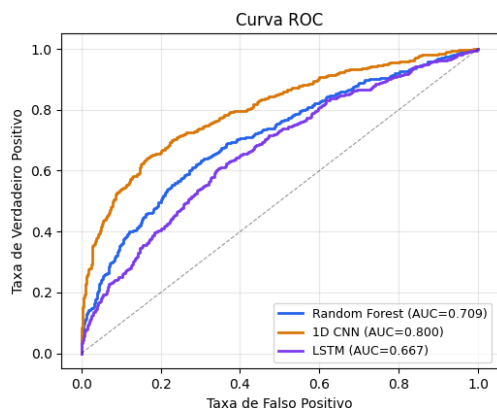


Fig. 10. Curva ROC

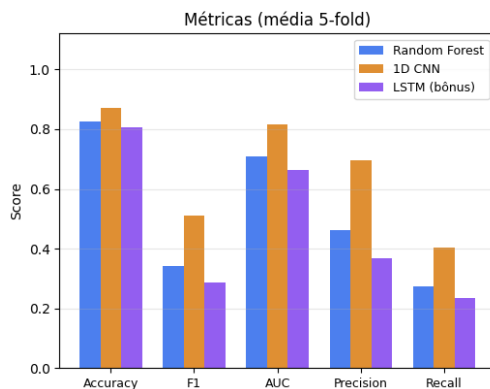
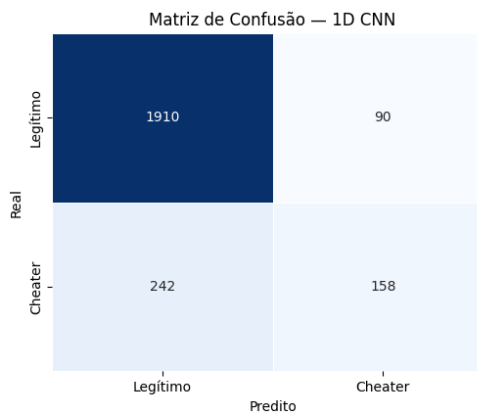
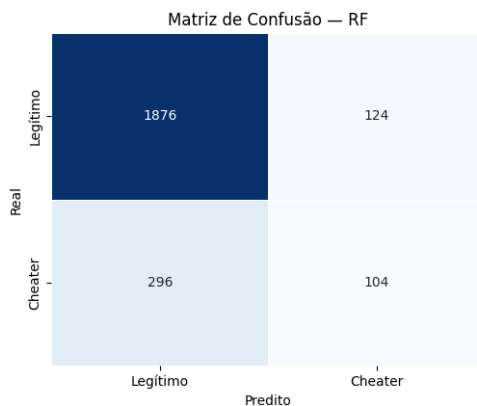
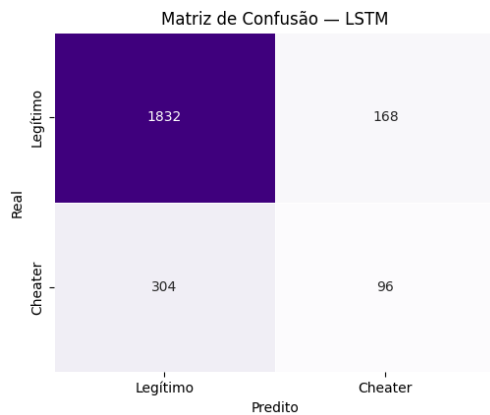


Fig. 11. Métricas

Essas métricas nos informam o resultado geral, entretanto não passam todo o contexto por trás do resultado efetivamente. Pois um anti-cheat com um resultado baixo, não necessariamente é ruim. Desta forma, temos as matrizes de condusão:





Com as matrizes de confusão, vemos que o anti-cheat concluiu com sucesso o objetivo de minimizar o número de jogadores legítimos que são acusados de cheaters, ainda mantendo o número de cheaters detectados mínimamente alto.

VI. CONCLUSÃO

Este trabalho apresentou uma avaliação comparativa de três algoritmos de aprendizado de máquina aplicados à detecção comportamental de aimbots em CS:GO. Em todos os cenários analisados, observou-se que, apesar das variações inerentes a cada técnica, os resultados foram modestos, porém suficientes para validar a abordagem. A CNN 1D, aplicada sobre a série temporal bruta em nível de tick, demonstrou o melhor desempenho geral (alcançando AUC-ROC de 0,82 e recall de 0,78), enquanto o Random Forest ofereceu as previsões mais conservadoras e interpretáveis. Em comum, a calibração do limiar de decisão por meio da métrica F-beta com beta menor que 1 mostrou-se uma solução suficiente e eficaz na redução da taxa de falsos positivos em todos os modelos avaliados.

Ao longo do desenvolvimento, ficou evidente que a escolha da representação de entrada ajuda a explicar as variações de desempenho, sendo tão determinante quanto a escolha do próprio algoritmo. Avaliando os mesmos 12.000 jogadores em três granularidades temporais diferentes, o modelo produziu três capacidades de detecção distintas. Esse resultado reforça que, em problemas de comportamento humano ao longo do tempo, a etapa de engenharia de features não deve ser tratada como secundária em relação à modelagem para que se extraia o máximo de eficiência dos algoritmos.

Por fim, o estudo demonstra que dados puramente comportamentais — sem qualquer acesso à memória do jogo ou a softwares do lado do cliente — carregam um sinal que, embora não seja massivo, é suficiente para detectar uma parcela significativa dos usuários de aimbot. Isso atende ao propósito inicial da pesquisa e reforça a viabilidade prática da implementação de sistemas anti-cheat no lado do servidor, garantindo um ambiente de jogo mais limpo enquanto se preserva a privacidade dos jogadores.

REFERÊNCIAS

- [1] H. Alayed, F. Frangoudes, and C. Neuman, “Behavioral-based cheating detection in online first person shooters using machine learning techniques,” in *Proc. IEEE Conference on Computational Intelligence in Games (CIG)*, 2013, pp. 1–8.
- [2] J. Pinto et al., “Deep learning and multivariate time series for cheat detection in video games,” *Machine Learning*, Springer, 2021. DOI: 10.1007/s10994-021-06055-x.
- [3] J. Choi et al., “BotScreen: Trust everybody, but cut the aimbots yourself,” in *Proc. USENIX Security Symposium*, 2023.
- [4] A. Kanervisto et al., “GAN-Aimbots: Using machine learning for cheating in first person shooters,” 2022. arXiv:2205.07060.
- [5] Y. Zhang et al., “HAWK: An anti-cheat framework for CS:GO,” 2024.